# API Security Risk Analysis Report

## Future Interns – Cyber Security Internship Task 3


### Submitted By

**Maanas Sri Udbhav Maddula**


### Project Title

**API Security Risk Analysis of JSONPlaceholder Public API**

### Submission Date

June 2026

# Table of Contents



# 1. Introduction

Application Programming Interfaces (APIs) are the backbone of modern digital services. APIs enable communication between web applications, mobile applications, cloud platforms, and

third-party services. Due to their widespread use, APIs have become attractive targets for attackers seeking unauthorized access to sensitive information.

This project focuses on conducting a read-only API Security Risk Analysis of a publicly available testing API. The objective is to identify potential security weaknesses, assess their business impact, and provide remediation recommendations without performing any exploitation activities.

The assessment was conducted ethically using publicly accessible endpoints and standard API testing techniques.

# 2. Objectives

The primary objectives of this assessment were:

- Analyze public API endpoints
- Review authentication requirements
- Inspect API request and response headers
- Examine exposed data in responses
- Identify potential API security risks
- Classify risks based on severity
- Assess business impact
- Provide remediation recommendations

# 3. API Overview

### API Name

JSONPlaceholder

### Base URL

https://jsonplaceholder.typicode.com

### Description

JSONPlaceholder is a free online REST API designed for testing and learning purposes. It provides mock data and supports common HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

### Endpoints Tested

- GET /users
- GET /posts
- GET /comments
- GET /users/1
- POST /users

# 4. Scope and Ethical Considerations

## In Scope

- Public API testing
- Read-only analysis
- Response inspection
- Header analysis
- Risk identification
- Documentation review

## Out of Scope

- Exploitation attempts
- Authentication bypass
- Denial of Service testing
- Unauthorized access attempts
- Production system testing

All testing activities were conducted ethically and within the allowed scope.

# 5. Tools Used

### Postman

Used to send HTTP requests and inspect API responses.

### Browser

Used for documentation review and endpoint verification.

### JSONPlaceholder Documentation

Used to understand API functionality and endpoint behavior.

# 6. Methodology

The assessment followed a structured approach:

### Step 1: API Documentation Review

The API documentation was reviewed to understand available endpoints and supported HTTP methods.

### Step 2: Endpoint Enumeration

Multiple endpoints were identified for testing purposes.

### Step 3: Request Analysis

GET and POST requests were sent using Postman.

### Step 4: Response Inspection

Returned data was reviewed for sensitive information exposure.

### Step 5: Header Inspection

Request headers were examined to identify authentication mechanisms.

### Step 6: Risk Assessment

Potential security concerns were identified and classified.

### Step 7: Recommendations

Remediation measures were proposed for each finding.

# 7. Endpoint Analysis

# Endpoint 1: GET /users

## Purpose

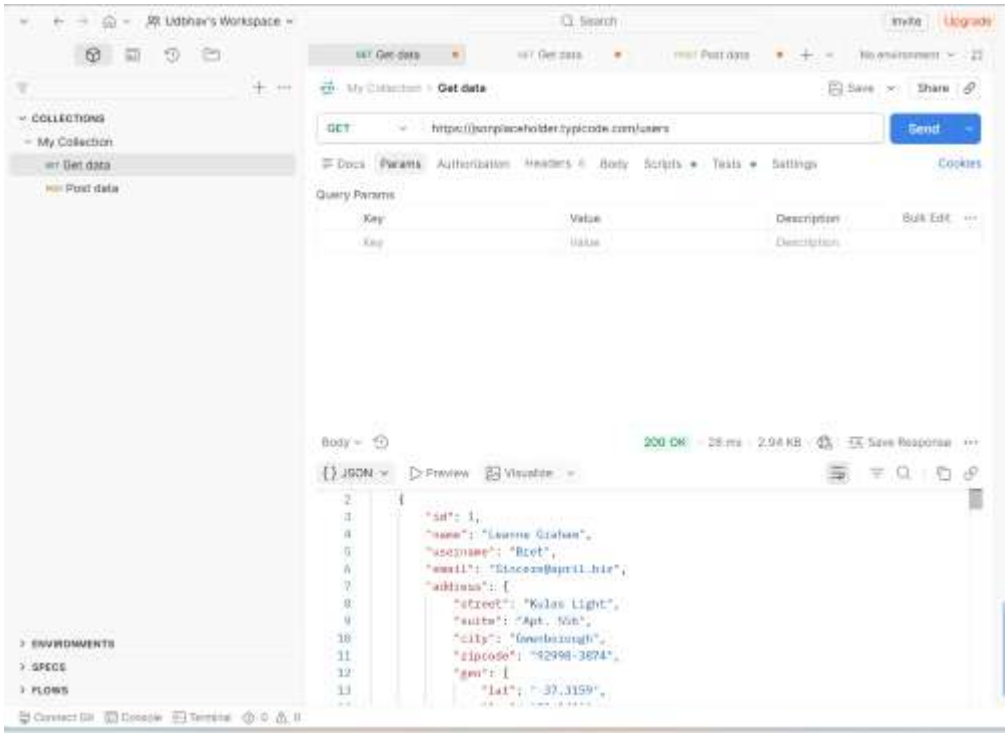
Retrieves user information.

## Observation

The endpoint returned:

- Name
- Username
- Email Address
- Phone Number
- Website
- Address
- Geographic Coordinates
- Company Information

## Evidence



# Endpoint 2: GET /posts

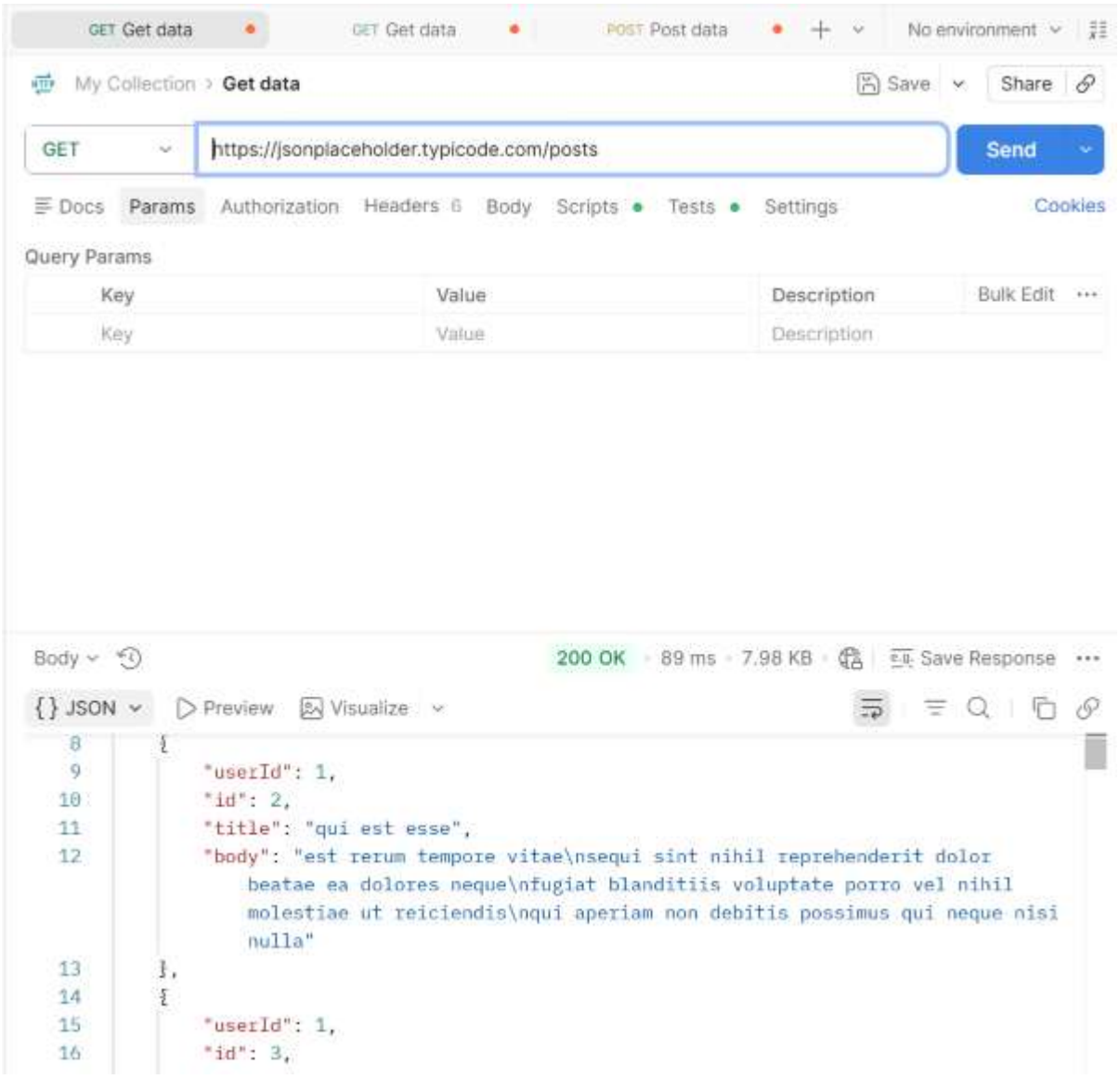
## Purpose

Retrieves post information.

## Observation

The endpoint returned post records without requiring authentication.

## Evidence



# Endpoint 3: GET /users/1

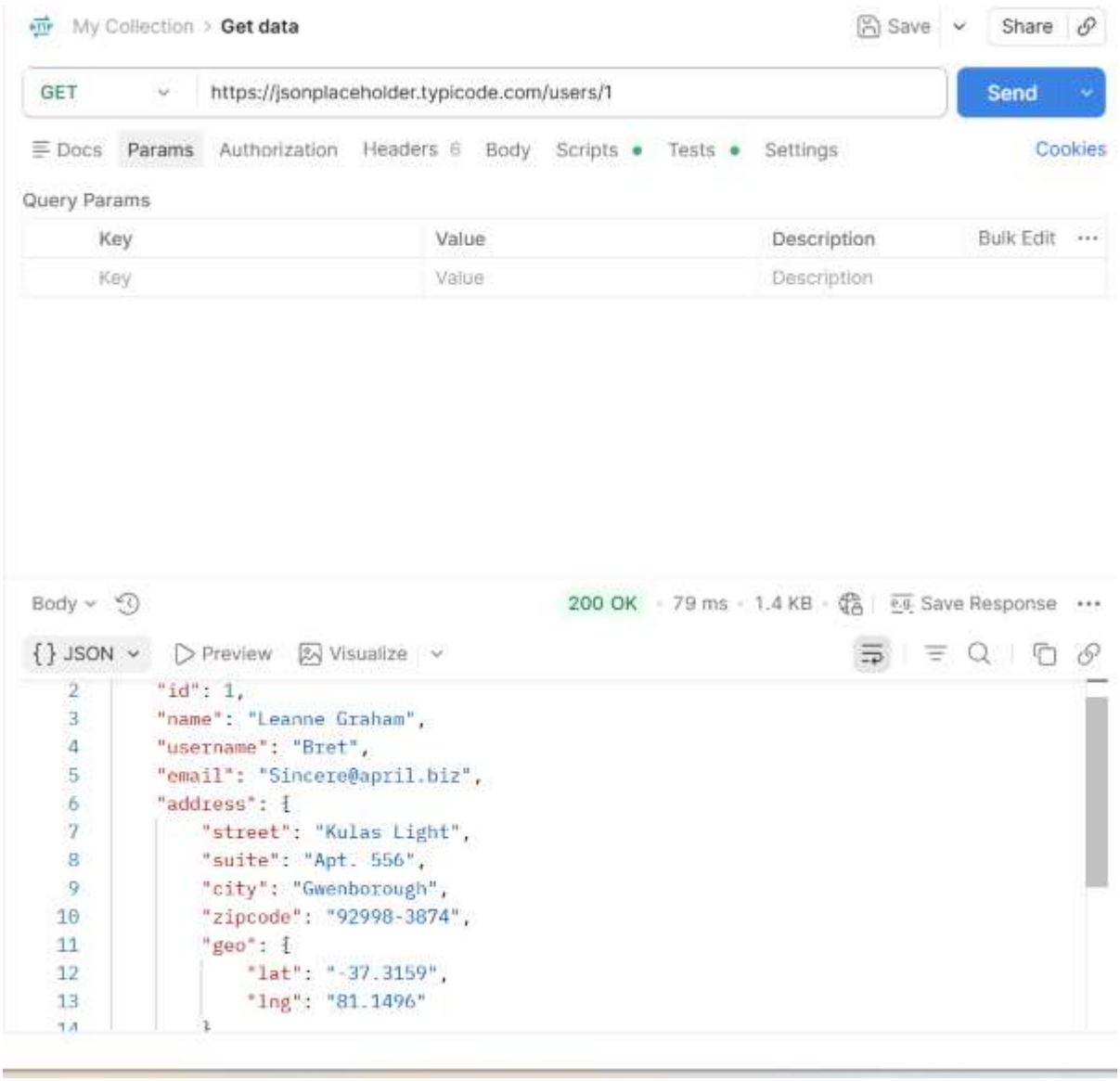
## Purpose

Retrieves information for a specific user.

## Observation

Individual records can be accessed using sequential numeric identifiers.

## Evidence




# Endpoint 4: Header Inspection

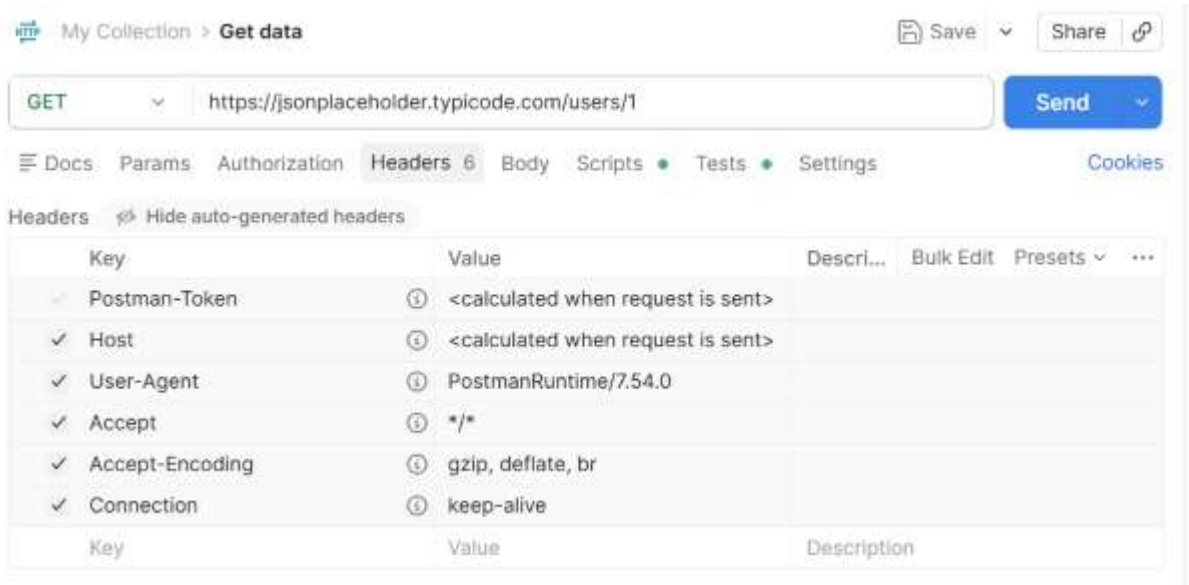
## Purpose

Review request configuration.

## Observation

No authentication tokens or API keys were present.

### Evidence




# Endpoint 5: GET /comments

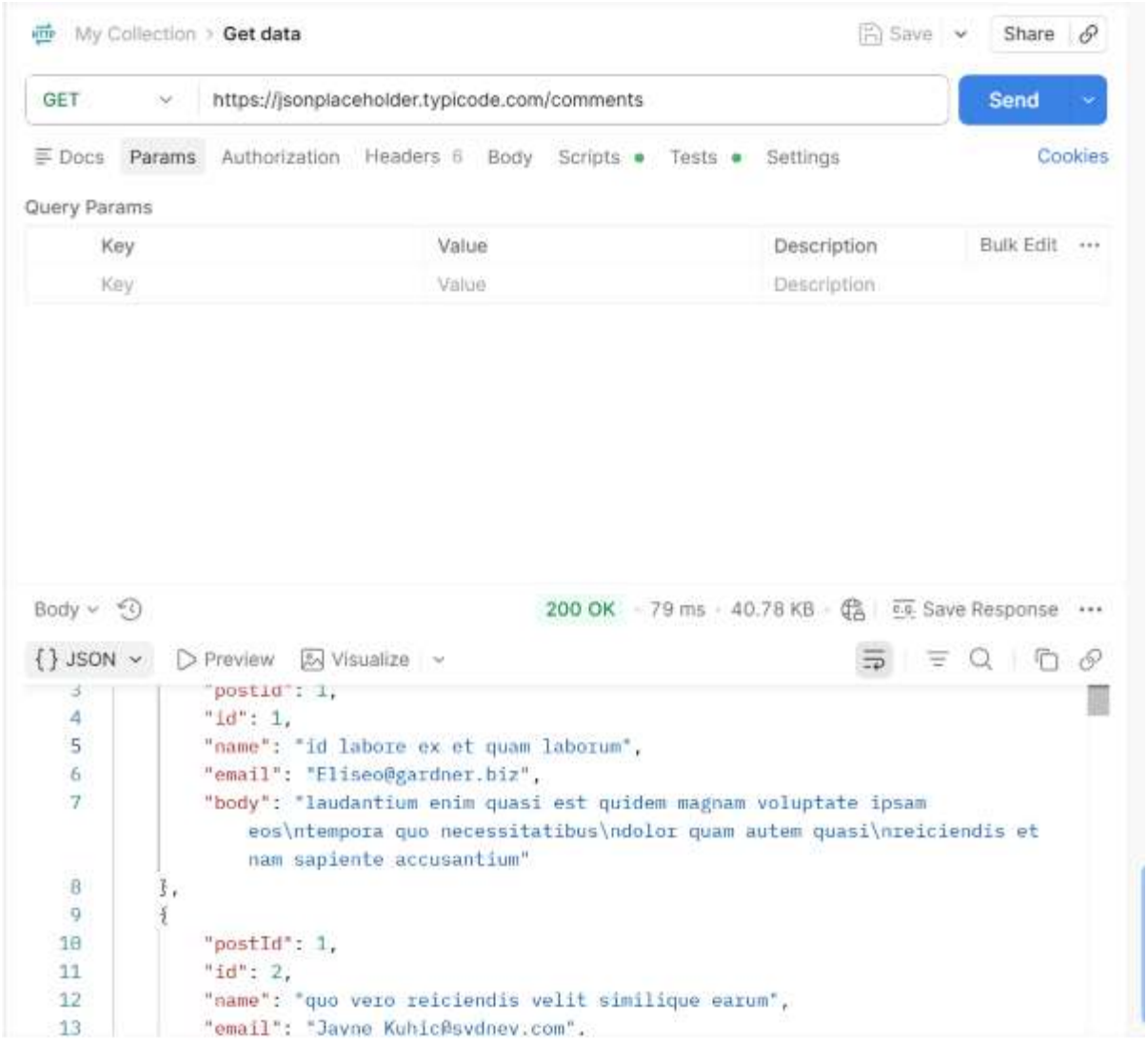
### Purpose

Retrieves comment information.

### Observation

Email addresses and user-generated content were exposed.

### Evidence




# Endpoint 6: POST /users

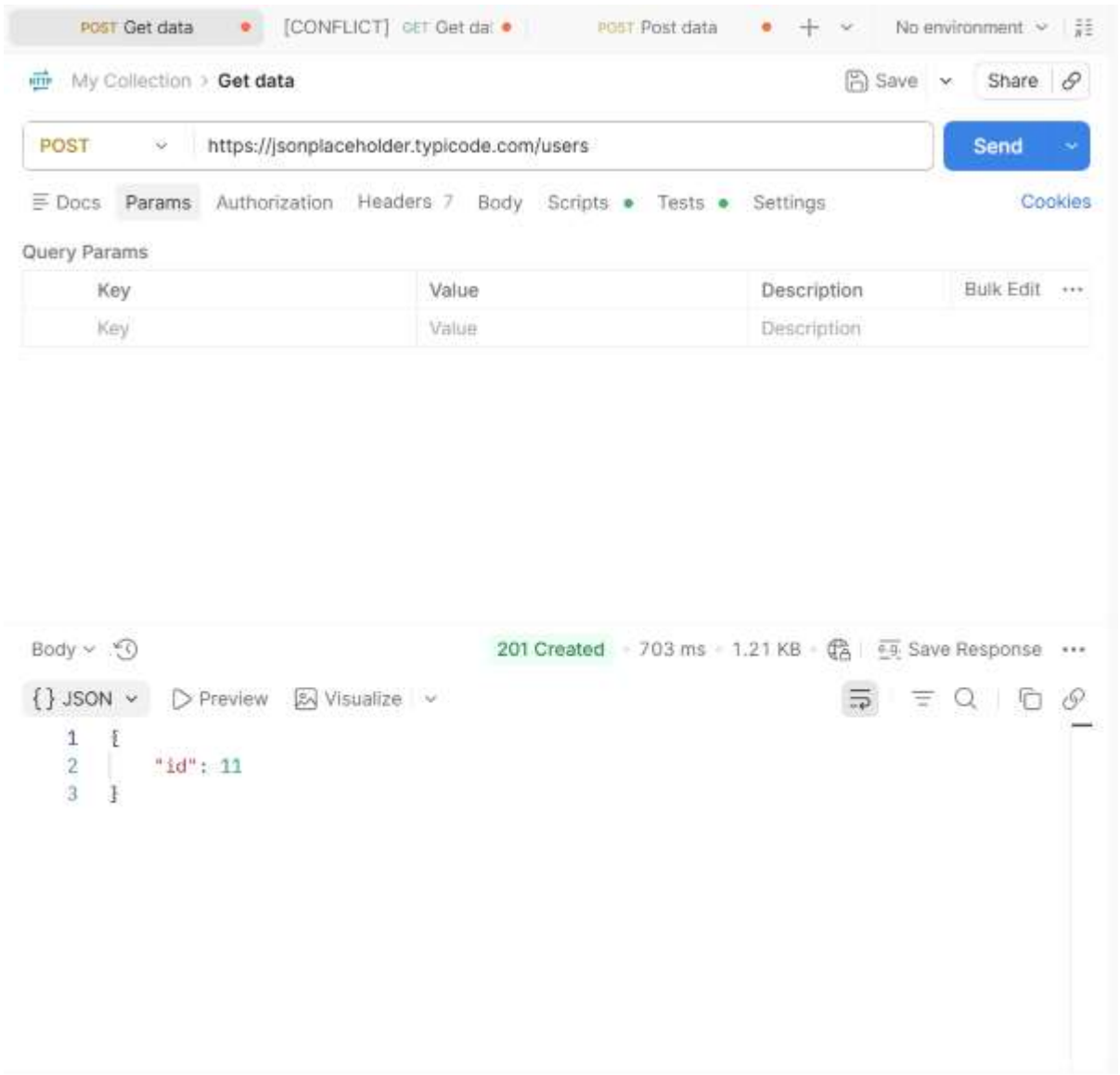
## Purpose

Test endpoint behavior for resource creation.

## Observation

The API accepted POST requests and returned a 201 Created response without authentication.

## Evidence




# 8. Security Findings

## Finding 1: Unauthenticated API Access

### Severity

Medium

### Description

Multiple endpoints were accessible without requiring authentication.

### Business Impact

Unauthorized users could access sensitive information in a production environment.

### Recommendation

Implement API Keys, JWT, or OAuth 2.0.

## Finding 2: Excessive Data Exposure

### Severity

Medium

### Description

User records contained extensive information including addresses, contact details, and geographic data.

### Business Impact

Exposed information may assist attackers in reconnaissance and social engineering attacks.

### Recommendation

Return only necessary data fields.

## Finding 3: Predictable Resource IDs

### Severity

Medium

### Description

User records could be accessed using sequential numeric identifiers.

### Business Impact

Attackers may enumerate records and gather information from multiple accounts.

### Recommendation

Use UUIDs and authorization controls.

## Finding 4: Missing Authentication Headers

### Severity

Low

### Description

Requests did not require authentication tokens.

### Business Impact

Weak access control increases exposure risks.

### Recommendation

Require authentication headers.

## Finding 5: Missing Rate Limiting Controls

### Severity

Low

### Description

No visible rate limiting mechanisms were observed.

### Business Impact

Attackers may abuse endpoints through automated requests.

### Recommendation

Implement throttling and request limits.

## Finding 6: Unauthenticated POST Requests

### Severity

Medium

### Description

POST requests were accepted without authentication.

### Business Impact

Attackers could create unauthorized records in production systems.

### Recommendation

Apply authentication and authorization controls.

# 9. Risk Assessment Summary

| Finding | Severity |
|---|---|
| Unauthenticated Access | Medium |
| Excessive Data Exposure | Medium |
| Predictable Resource IDs | Medium |
| Missing Authentication Headers | Low |
| Missing Rate Limiting | Low |
| Unauthenticated POST Requests | Medium |

# 10. Business Impact Analysis

If these findings existed in a production environment, organizations could face:

- Data Exposure
- Privacy Violations
- Unauthorized Data Access
- User Enumeration
- API Abuse
- Reputation Damage
- Regulatory Compliance Issues

The impact could range from operational disruptions to significant financial losses.

# 11. Remediation Recommendations

### Authentication

- Implement JWT
- Use OAuth 2.0
- Require API Keys

### Authorization

- Apply Role-Based Access Control
- Validate resource ownership

### Data Protection

- Return only required fields
- Remove unnecessary information

### Rate Limiting

- Limit requests per minute
- Detect abnormal activity

### Monitoring

- Log API activity
- Monitor suspicious behavior

# 12. Best Practices for API Security

- Follow OWASP API Security Top 10
- Enforce strong authentication
- Implement authorization checks
- Validate input data
- Encrypt sensitive information
- Monitor API usage
- Conduct regular security assessments

# 13. Conclusion

This API Security Risk Analysis successfully evaluated the JSONPlaceholder public API using ethical and read-only testing techniques. Several observations were identified, including unauthenticated access, excessive data exposure, predictable resource identifiers, and lack of visible rate limiting controls.

Although JSONPlaceholder is intentionally designed as a public testing API, these findings demonstrate security concerns that would require remediation in a production environment. Organizations should implement strong authentication, authorization, monitoring, and data protection controls to ensure secure API operations.

# 14. References


- OWASP API Security Top 10
- JSONPlaceholder Documentation
- Postman Documentation
- API Security Checklist
- Public APIs Collection Repository